Robotique
CRC
Robotics

CRC

COMPÉTITION DE PROGRAMMATION



LIVRET DE PROBLÈMES

BLOC B

QUELQUES NOTES

- Les règles complètes sont dans la section 4 du livret des règlements
- Vous avez jusqu'au **14h29** pour remettre vos solutions
- N'hésitez pas à nous poser des questions et à communiquer avec les membres de votre équipe de programmation. Ce n'est pas un examen!
- **On vous donne des fichiers modèles faciles à utiliser pour votre code. Merci de les utiliser.**

UTILISATION DU FICHIER MODÈLE

- Tous les fichiers possèdent une fonction solve() dans lequel vous devez résoudre le problème. **NE CHANGEZ PAS LE NOM DE CETTE FONCTION!**
- Pour rouler vos tests, dans un terminal (nous vous conseillons l'utilisation de vscode) écrivez la ligne de commande: `pytest` pour rouler les tests. Si vous faites la commande `pytest` dans le répertoire contenant tous les fichiers, tous les tests seront exécutés.
- Pour rouler les tests d'un problème spécifique, naviguez dans le répertoire de la question et exécutez la commande `pytest` pour rouler les tests du problème. Vous pouvez faire la même chose pour une section entière!

STRUCTURE

Chaque problème contient une petite mise en situation comme celle-ci expliquant les fondements du problème et donnant les bases nécessaires pour résoudre celui-ci. Chaque problème préparatoire contient aussi la répartition des points pour le problème.

Spécification d'entrée et de sortie:

Dans cette section, on retrouve les caractéristiques des entrées qui peuvent être fournies au code en question ainsi que les critères attendus pour les sorties du programme.

Exemple d'entrée et de sortie:

Dans cette section se trouve un exemple d'entrée (parfois constitué lui-même de plusieurs sous-exemples) pour que vous puissiez tester votre code. L'exemple de sortie donne donc la réponse attendue pour cette entrée.

Explication de la première sortie:

Si le problème n'est toujours pas clair après la mise en situation, l'explication de la première sortie sert parfois à démêler le tout en expliquant comment la première entrée est traitée et en montrant le chemin menant à cette réponse.

Table des matières

Échec et Mat (CM) 4

CM1: Des dames partout (50 points) 4

CM2: La marche du cavalier (70 points) 5

Les 7 merveilles du monde (WW) 7

WW1: Pyramide de Khéops (35 points) 7

WW2: Jardins suspendus de Babylone (15 points) 9

WW3: Statue de Zeus (50 points) 11

WW4: Temple d'Artémis (30 points) 13

WW5: Mausolée d'Halicarnasse (15 points) 15

WW6: Colosse de Rhodes (40 points) 16

WW7: Phare d'Alexandrie (85 points) 18

Mastermind (MM) 21

MM: Master of the mind (100 points) 21

Échec et Mat (CM)

CM1: Des dames partout (50 points)

Vous allez devoir placer des dames sur un jeu d'échecs égal à la dimension du côté du carré. Aucune des dames ne doit voir une autre dame. Les dames dans un jeu d'échecs peuvent se déplacer sur une ligne d'autant de cases possibles, sur une rangée ou en diagonale et aucune dame ne doit pouvoir en voir une autre. Bref, si vous avez une grille 4x4, vous devrez y mettre 4 dames qui ne se voient pas les unes les autres.

Spécifications d'entrée:

En entrée vous recevrez:

- ***size***: longueur d'un des côtés du plateau et aussi le nombre de dames que vous devrez mettre

Spécifications de sortie:

En sortie vous devrez fournir:

- un array de array de 2 int représentant les positions des dames sur le plateau

Exemple d'entrée:

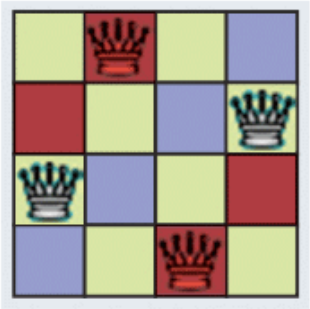
```
4
8
9
```

Exemple de sortie:

```
Il y a plusieurs réponses possibles pour une même entrée,
mais voici une sortie valide pour l'exemple 1
[[0, 1], [1, 3], [2, 0], [3, 2]]
```

Explication de la première sortie:

On remarque que chacune des reines n'en a aucune autre sur la même colonne, rangée ou diagonale. Aucune d'entre elles ne se voient.



CM2: La marche du cavalier (70 points)

Un cavalier (provenant des échecs, bien sûr) bouge comme un cavalier aux échecs sur un plateau d'échecs de 8 par 8 où les cases sont numérotées de 0 à 63 en serpentin ascendant:

63	62	61	60	59	58	57	56
48	49	50	51	52	53	54	55
47	46	45	44	43	42	41	40
32	33	34	35	36	37	38	39
31	30	29	28	27	26	25	24
16	17	18	19	20	21	22	23
15	14	13	12	11	10	9	8
0	1	2	3	4	5	6	7

Quand le cavalier bouge sur cet échiquier, il accumule des pièces Kryptik correspondant au numéro de la case sur laquelle il a atterri. Cependant, le cavalier combattant la richesse et étant complètement anti-matérialiste, il souhaite accumuler le moins de pièces Kryptik possible. Votre cavalier "atterrira" sur une case de départ (voulant dire qu'elle compte dans le total de pièces) et devra atteindre la case d'arrivée en accumulant le moins de pièces possibles. Vous devrez donner le total de pièces accumulées pour la trajectoire optimale.

Spécifications d'entrée:

En entrée vous recevrez:

- ***start***: un entier correspondant au numéro de la case initiale du cavalier
- ***end***: un entier correspondant au numéro de la case d'arrivée du cavalier

Spécifications de sortie:

En sortie vous devrez fournir:

- un entier donnant le nombre de pièces Kryptik accumulées pour une trajectoire optimale

Exemple d'entrée:

```
start: 35          end: 34
start: 23          end: 37
start: 0       end: 63
start: 50          end: 55
```

Exemple de sortie:

```
114
93
167
182
```

Explication de la première sortie:

Nous ne comparerons pas tous les chemins possibles comme il y en a énormément. Il y a deux chemins optimaux qui partent de 35 et se rendent à 34. Le premier passe par les cases 35-26-19-34 et le deuxième, par 35-18-27-34. La somme de n'importe lequel de ces chemins est de 114.

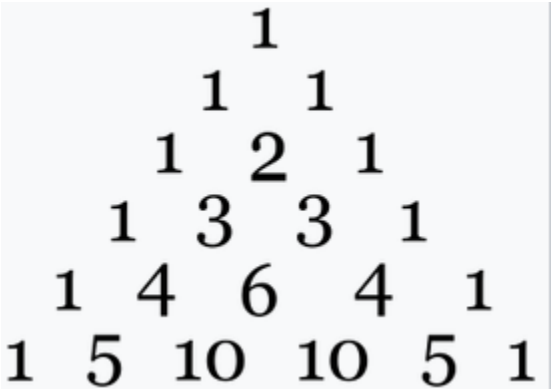
Note en bas de page:

Saviez-vous qu'on peut placer jusqu'à 32 cavaliers sur un échiquier sans qu'aucun puisse se voir?

Les 7 merveilles du monde (WW)

WW1: Pyramide de Khéops (35 points)

Le triangle de Pascal est une construction de nombres particulière où on additionne les nombres d'un étage par paires pour trouver l'étage suivant, en commençant par un simple 1. Voici ses six premiers étages:



La logique est que chaque nombre se "transmet" aux deux immédiatement en diagonale en dessous. C'est aussi comme si on avait des 0 invisibles entourant chaque étage, expliquant pourquoi chaque étage est toujours encadré de 1. Regardons le passage de l'étage 5 à l'étage 6, par exemple. 0+1 = 1, 1+4=5, 4+6=10, 6+4=10, 4+1=5 et 1+0=1, ce qui correspond bien au sixième étage.

Nous vous demanderons d'effectuer le même principe, mais sur une pyramide en trois dimensions. Dans cette situation, chaque nombre d'un étage au dessus se transmet aux 4 nombres immédiatement en diagonale:



Voici les trois premiers étages d'une telle pyramide. En passant de l'étage 2 à l'étage 3 et en considérant aussi les 0 "invisibles" tout autour, on obtient 0+0+0+1=1, 0+0+1+1=2, 0+0+1+0=1, 0+1+0+1=2, 1+1+1+1=4, 1+0+1+0=2, 0+1+0+0=1, 1+1+0+0=2 et 1+0+0+0=1, ce qui complète le troisième étage. Le principe est le même peu importe le niveau de la pyramide. Naturellement, ceci fait en sorte que chaque étage *n* est toujours un carré de *n* par *n* avec des 1 aux quatre coins.

Spécifications d'entrée:

En entrée vous recevrez:

- ***level:*** un entier représentant l'étage à afficher

Spécifications de sortie:

En sortie vous devrez fournir:

- un tableau 2D d'entiers affichant l'étage spécifique de la pyramide

Exemple d'entrée:

```
3
2
5
7
```

Exemple de sortie:

```
[[1,2,1],[2,4,2],[1,2,1]]
[[1,1],[1,1]]
[[1,4,6,4,1],[4,16,24,16,4],[6,24,36,24,6],[4,16,24,16,4],[1,4,6
,4,1]]
[[1,6,15,20,15,6,1],[6,36,90,120,90,36,6],
[15,90,225,300,225,90,15],[20,120,300,400,300,120,20],
[15,90,225,300,225,90,15],[6,36,90,120,90,36,6],
[1,6,15,20,15,6,1]]
```

Explication de la première sortie:

Voir la description du problème pour voir comment on est passés du deuxième au troisième étage. Pour passer du premier au deuxième étage, il suffit de transmettre le 1 initial à ses 4 voisins en dessous.

Note en bas de page:

Aller voir plus d'informations sur le triangle de Pascal pour découvrir plein d'applications intéressantes dans le domaine des sciences.

WW2: Jardins suspendus de Babylone (15 points)

Les jardins suspendus de Babylone contiennent une énorme diversité de plantes qui coexistent. Pour souligner la visite d'un roi venant d'un royaume lointain, les jardiniers de Babylone ont décidé de décorer l'entrée principale des jardins avec un échantillon des meilleures plantes que le roi pourra observer lors de sa visite des lieux. Cette tâche pose par contre un petit problème: les différentes espèces de plantes à fleurs ne possèdent pas toutes le même temps de floraison. L'équipe qui s'occupe de la décoration devra donc faire bien attention de s'assurer que toutes les plantes fleurissent au bon moment.

Nous aurons donc besoin d'un calendrier qui inclut quand chaque sorte devra être plantée. Ainsi le calendrier commence douze mois avant la visite, l'élément 0 doit contenir les plantes devant fleurir en douze mois, celui à l'élément 1 prend 11 mois à fleurir et ainsi de suite.

Si jamais deux plantes ont le même temps avant de fleurir, elles doivent être en ordre alphabétique dans le calendrier avec un espace entre elles dans l'affichage.

Spécifications d'entrée:

En entrée vous recevrez:

- ***flowers***: un array contenant des array de deux string, une pour le nom de plante et l'autre pour le temps de fleurissement

Spécifications de sortie:

En sortie vous devrez fournir:

- un array contenant une string par mois des plantes ayant besoin d'être plantées

Exemple d'entrée:

```
[['rose', '8'],
['tulip', '6'],
['petunia', '4'],
['sunflower', '5'],
['orchid', '4'],
['lily', '11'],
['iris', '8']]

[['sunflower', '5'],
['petunia', '4'],
['lily', '11'],
['geranium', '7'],
['daffodil', '12'],
['lavender', '2']]

[['lily', '11'],
['geranium', '7'],
['daffodil', '12'],
```

```
['iris', '8'],
['lavender', '2']]
```

Exemple de sortie:

```
['', 'lily', '', '', 'iris rose', '', 'tulip', 'sunflower',
'orchid petunia', '', '', '']

['daffodil', 'lily', '', '', '', 'geranium', '', 'sunflower',
'petunia', '', 'lavender', '']

['daffodil', 'lily', '', '', 'iris', 'geranium', '', '', '',
'', 'lavender', '']
```

Explication de la première sortie:

Pour le premier exemple, on a aucune fleur qui demande 12 mois, seulement les lily pour 11 mois. Rien pour 10 et 9 mois. Pour 8 mois nous avons iris et rose séparés par un espace et mis en ordre alphabétique. Il n'y a pas de fleurs pour 7 mois d'avance, puis il y a les tulipes 6 mois d'avance, les tournesols 5 mois d'avance. Finalement les orchid et petunia 4 mois d'avance et aucune fleur pour les 3 derniers mois.

WW3: Statue de Zeus (50 points)

Lors d'un voyage avec son école dans la ville d'Olympie, Claire Éclairé suit son esprit d'entreprenariat et demande à ses amies combien d'argent elles pourraient générer si elles "recyclaient" les matériaux de à la statue de Zeus. Ne pouvant pas dormir la nuit avec la question qui leur trottait dans la tête, le groupe décide d'aller récolter des données pour mettre fin à leur angoisse.

Elles commencent par se rendre à la statue pour mesurer les dimensions du brut à partir duquel la statue a été fabriquée (si c'est un anachronisme, elles ont une machine à remonter dans le temps). Avant de commencer à sculpter la statue, Phidias (l'artiste ayant créé cette merveille du monde) n'avait qu'un gros prisme à base carré dans son atelier. Armé d'un ruban à mesurer, notre groupe d'étudiantes réussit à mesurer la longueur d'un des côtés de la base de ce bloc. Elles sortent ensuite leur équipement de spectrométrie de masse pour déterminer le pourcentage d'ivoire et d'or présents dans la statue (% ivoire + % or = 100%). Le groupe se dirige ensuite vers la bibliothèque pour faire de la recherche. Elles trouvent d'abord la masse volumique de l'ivoire (ρ/roh) avec une simple recherche sur internet.

Elles plongent ensuite dans des livres couverts de poussière pour lire les notes originales de l'artiste. Elles y trouvent la masse du prisme à base carré qui a été utilisé (brut) et le pourcentage du volume qui a été enlevé lors de la création de cette énorme œuvre d'art. Satisfaites de l'information qu'elles ont récolté, les filles retournent à leur chambre d'hôtel pour préparer le PowerPoint qu'elles veulent présenter à de potentiels investisseurs. Leur plan est de "réutiliser" l'or de la statue pour en faire des boules de Noël en or pur. Elles feront donc fondre l'or pour couler des boules de rayon '**r**' qu'elles vendront ensuite au prix de vente **p**. Elles feront évidemment un contrôle de qualité pour s'assurer que seules les boules complètes sont vendues.

Si elles vendaient des boules non-complètes, leur image publique serait ruinée avant même qu'elles puissent se faire un nom dans le domaine des décorations de sapin de luxe. À la dernière diapositive de leur présentation audio-visuelle, elles présentent le profit estimé que leur projet générera si elles "oublient" de payer leur main-d'œuvre. Veuillez nous donner ce profit comme sortie.

ordre d'opération pour créer la statue:
-brut
-enlever matière -> portion ivoire finie
-appliquer l'or
-Voila!

Spécifications d'entrée:

En entrée vous recevrez:

- ***l:*** *longueur coté*
- ***i:*** *pourcentage d'ivoire dans la statue (0-100)*
- ***rohI:*** *masse volumique ivoire*
- ***rohO:*** *masse volumique or*
- ***m:*** *masse du brut*
- ***e:*** *pourcentage de volume enlevé du brut pour faire la statue (0-100)*

- ***r:*** *rayon des sphères à fabriquer*
- ***p:*** *prix de vente d'une sphère*

Spécifications de sortie:

En sortie vous devrez fournir:

- un float: le montant amassé après avoir vendu toutes les sphères

Exemple d'entrée:

```
l: 6, i: 7, rohI: 8, rohO: 8.5, m: 9, e: 10, r: 0.5, p: 3
l: 40, i: 16, rohI: 8, rohO: 10, m: 200, e: 80, r: 1, p: 8
l: 33, i: 66, rohI: 0.9, rohO: 17, m: 666, e: 50, r: 3, p: 1000
l: 24, i: 90, rohI: 18, rohO: 1, m: 10, e: 60, r: 0.2, p: 3090
```

Exemple de sortie:

```
6
160
0
58710
```

Explication de la première sortie:

Vous devez trouver la démarche pour trouver le profit par vous même et la présenter en sortie.

WW4: Temple d'Artémis (30 points)

Le temple d'Artémis contient de multiples colonnes qui sont tombées au cours de sa destruction. En tant qu'archéologue, vous souhaitez calculer le nombre de colonnes nécessaires pour soutenir le toit du temple selon certaines dimensions. Tout le tour du bâtiment est composé d'une rangée de colonnes, sauf la façade avant contient deux rangées de colonnes. Vous recevrez la longueur et la largeur du bâtiment en nombre de colonnes.



Spécifications d'entrée:

En entrée vous recevrez:

- ***length***: un entier représentant la longueur du bâtiment en terme de colonnes
- ***width***: un entier représentant la largeur du bâtiment en terme de colonnes

Spécifications de sortie:

En sortie vous devrez fournir:

- un entier étant le nombre de colonnes du temple

Exemple d'entrée:

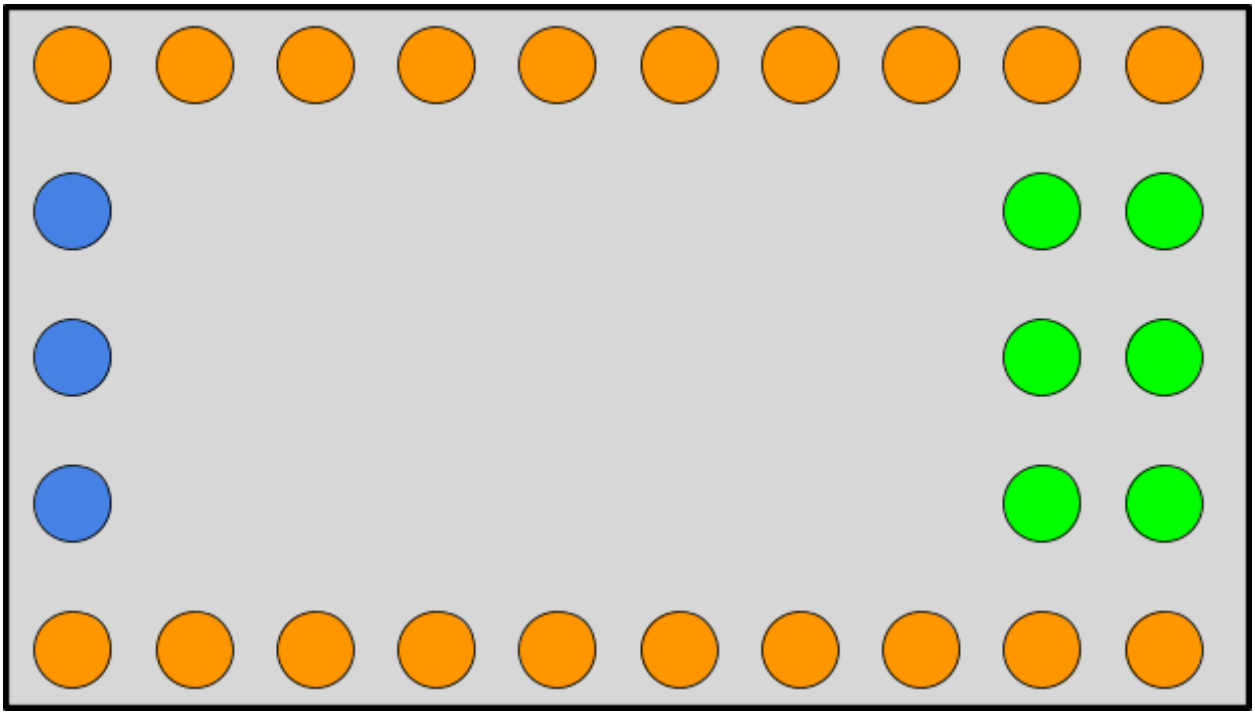
```
length: 10 width: 5
length: 20 width: 10
length: 11 width: 7
```

Exemple de sortie:

```
29
64
37
```

Explication de la première sortie:

Pour un temple ayant une longueur de 10 colonnes et une largeur de 5 colonnes, nous avons un total de 29 colonnes. Commençons par séparer le problème en regardant différentes sections de colonnes. Tout d'abord, sur le côté du temple nous avons deux rangées de 10 colonnes ce qui nous fait 20 colonnes (orange). Puis en arrière nous avons 5 colonnes. Par contre, attention, 2 de ces 5 colonnes ont déjà été comptées dans les colonnes du côté du temple donc il n'y en a que 3 (bleu). En avant du temple nous avons 2 rangées de 5 colonnes donc 10 colonnes, mais il ne faut pas oublier que 4 d'entre elles ont déjà été comptées dans le calcul des colonnes du côté. Ce qui nous fait 6 colonnes en avant (vert). Nous avons donc 20 colonnes de côté + 3 autres en arrière du temple + 6 autres en avant du temple. 20 + 3 + 6 = 29



WW5: Mausolée d'Halicarnasse (15 points)

Lors de recherches archéologiques sur le mausolée, vous essayez de découvrir l'âge des corps enterrés dans les tombes du mausolée. Pour déterminer leur âge, vous décidez d'utiliser la datation grâce au carbone.

Pour ce faire, vous regardez la concentration actuelle du carbone 14 que vous devez comparer avec la concentration initiale du carbone 14 en utilisant la formule suivante. Ceci vous permet de savoir depuis combien de temps datent les tombes du mausolée. Comme les valeurs de concentration sont faibles, elles vous seront données par notation scientifique. Donc, la valeur et sa puissance de 10 par laquelle la multiplier.

$$time = \frac{1}{\lambda} ln(\frac{C_0}{C_t})$$

$$Where\ C_0 = 10^{-12} \quad and \quad \lambda = 0.00012$$

Une fois le temps découvert, comme nous sommes en 2024 il vous suffit de soustraire pour trouver l'année de création de la tombe.

Spécifications d'entrée:

En entrée vous recevrez:

- ***v***: un float représentant la valeur de C_t avant l'exposant
- ***ex***: un int de l'exposant appliqué à v pour obtenir C_t

Spécifications de sortie:

En sortie vous devrez fournir:

- un entier de la valeur arrondie du nombre d'années de la tombe

Exemple d'entrée:

```
v: 8.32  ex: -12
v: 8.26  ex: -12
v: 0.7999  ex: -11
```

Exemple de sortie:

```
491
431
163
```

Explication de la première sortie:

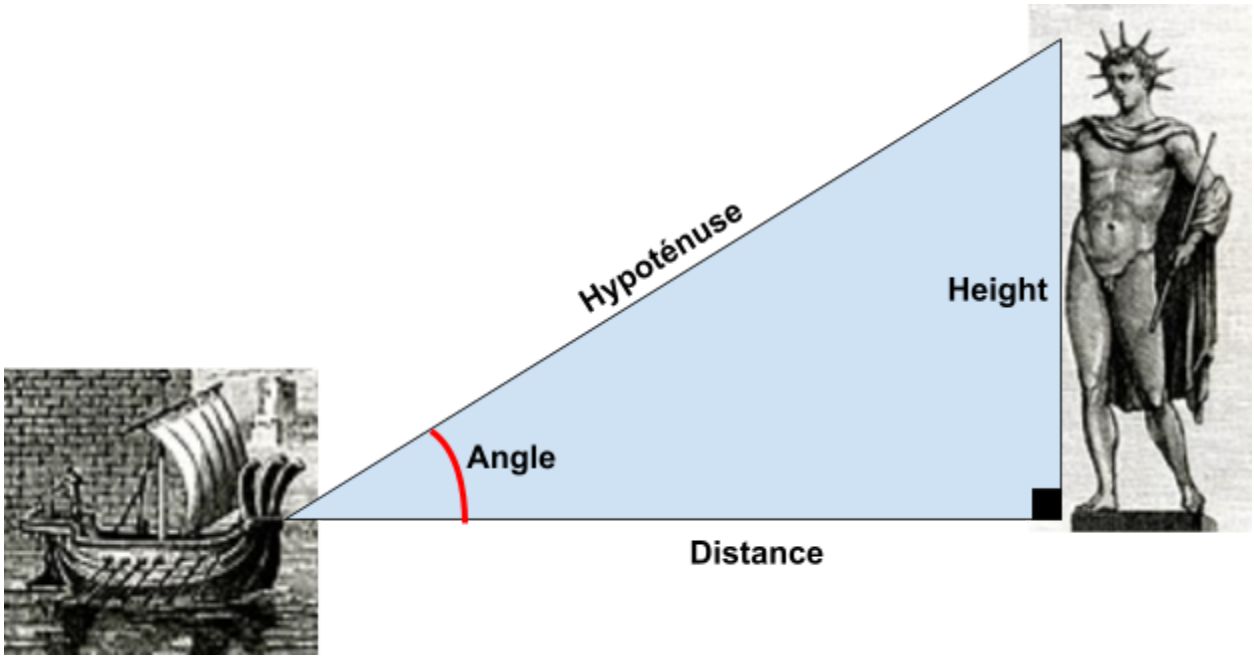
Pour le premier exemple, on prend la valeur de $C_t = 8.32 \cdot 10^{-12}$. On peut calculer une valeur de time équivalent à 1532.69031801. On peut arrondir le résultat pour obtenir un nombre d'années de 1533 et une fois qu'on soustrait ce nombre de 2024 pour obtenir l'année 491.

WW6: Colosse de Rhodes (40 points)

Vous êtes en bâteau, dans la grèce antique et vous voyez au loin la prestigieuse cité de Rhodes. À l'entrée de la ville se situe l'immense statue d'Hélios, le dieu du soleil. Cette statue est souvent nommée le colosse de Rhodes!

Avec l'aide de votre excellent sens de l'observation et d'un sextant, vous essayez d'estimer la hauteur de la statue. Vous recevez une distance approximative en mètres ainsi que l'angle approximatif entre les pieds et la tête de la statue à partir de votre navire. Vous devrez donner la plus petite hauteur possible, ainsi que la plus grande hauteur possible de la statue.

Vous devrez donc utiliser les propriétés des triangles rectangles pour trouver la hauteur à partir des mesures de distance et d'angle.



Voici un rappel de quelques formules utiles de trigonométrie. Avec un angle et la distance du pied de la statue nous pouvons trouver l'hypoténuse.

$$cos(Angle) = \frac{distance}{hypoténuse} \Rightarrow hypoténuse = \frac{distance}{cos(Angle)}$$

Ayant maintenant la distance et l'hypoténuse, nous pouvons maintenant appliquer la formule de pythagore pour obtenir la hauteur de la statue.

$$(hypoténuse)^2 = (distance)^2 + (height)^2$$

$$height = \sqrt{(hypoténuse)^2 - (distance)^2}$$

Spécifications d'entrée:

En entrée vous recevrez:

- ***distance***: un entier du nombre de mètre de distance
- ***distance_uncertainty*****:** un float du pourcentage d'erreur possible
- ***angle***: un entier (angle de 0 à 365 degrés)
- ***angle_uncertainty***: un entier du nombre de dégré d'incertitude (de 0 à 365 degrés)

Spécifications de sortie:

En sortie vous devrez fournir:

- un array contenant deux floats (3 digits après la virgule) représentant la borne inférieure et supérieure de la grandeur mesurée

Exemple d'entrée:

```
distance: 40  distance_uncertainty: 0.05
angle: 30 angle_uncertainty: 2

distance: 80  distance_uncertainty: 0.01
angle: 15 angle_uncertainty: 2

distance: 30  distance_uncertainty: 0.05
angle: 45 angle_uncertainty: 4
```

Exemple de sortie:

```
[32.684, 37.799]
[18.285, 24.703]
[24.775, 36.237]
```

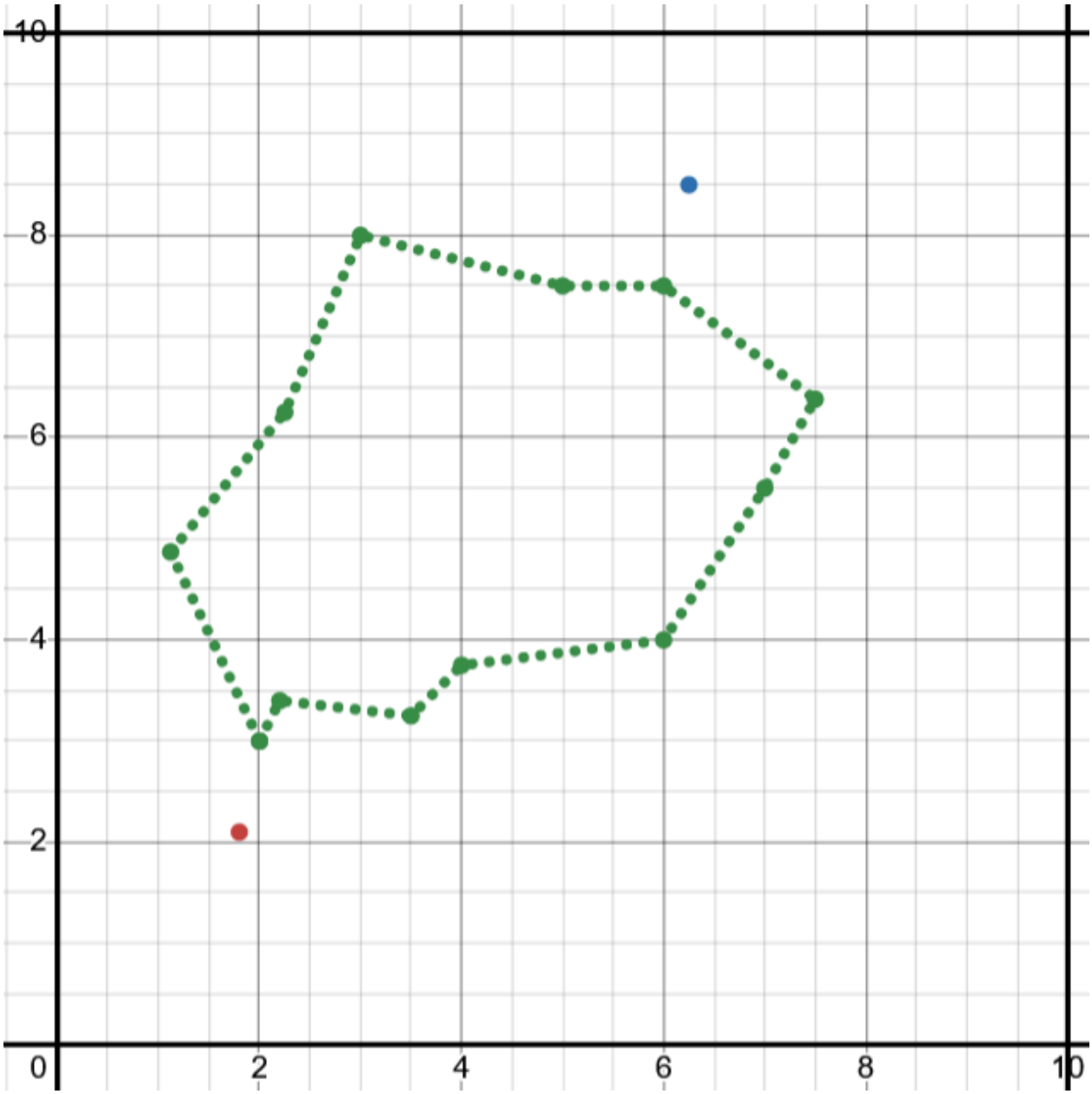
Explication de la première sortie:

Pour le premier exemple, on commence par calculer la borne inférieure de la grandeur de la statue. Pour ce faire, on suppose que l'angle est à sa borne la plus faible et que la distance est la plus petite aussi. On calcule la distance minimale en faisant 40m * (1 - 0.05) qui nous donne 38 mètres. Pour l'angle, on prend 30 - 2 donc 28 degrés. On peut maintenant calculer l'hypoténuse avec la formule donnée dans l'énoncé ce qui nous donne 43.0376619262 mètres. Nous pouvons appliquer le théorème de pythagore et trouver une hauteur de `32.6839462745` mètres. On arrondi donc à `32.684` mètres.

On applique les mêmes calculs avec comme distance maximale 42 mètres et 32 degrés. Ce qui nous donne une hypoténuse de 49.5254929412 mètres et une hauteur de 37.7991329407 qui peut être arrondi à 37.799 mètres.

WW7: Phare d'Alexandrie (85 points)

Une tempête s'annonce à l'horizon et une barque de pêcheurs tente de revenir à la côte saine et sauve. Le grand Phare d'Alexandrie peut les guider tant que sa lumière n'est pas complètement couverte par de grands nuages obscurs qui couvrent le ciel. Le problème se tiendra dans un plan cartésien de 10km par 10km. Les coordonnées du Phare et celles du bateau seront spécifiées pour chaque situation. **Une seule** série de points représentant un nuage sera donnée aussi en entrée. Ainsi, une situation typique peut ressembler à ceci:



Le nuage est en vert (nous avons relié les points pour mieux le voir), le Phare est en rouge et le bateau est en bleu. Le Phare émet de la lumière dans toutes les directions mais, évidemment, **la trajectoire de la lumière atteignant le bateau est une ligne droite directe entre le phare et le bateau.**

C'est là qu'on rencontre le problème avec les nuages: **l'intensité de la lumière provenant du Phare diminuera de 20% pour chaque km parcouru dans le nuage et ce, linéairement** (ce qui veut dire que 2 km parcourus font diminuer l'intensité de la lumière de 40%). Il sera utile de connaître ces notions de mathématiques:

- Deux points (x, y) suffisent à déterminer l'équation de droite reliant les deux points en déterminant la pente et l'ordonnée à l'origine
- On peut déterminer le point d'intersection de deux droites en en mettant deux équations de droites égales et en résolvant des deux côtés
- La distance entre deux points (x1, y1) et (x2, y2) se calcule avec la formule suivante:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

Si l'intensité de la lumière atteignant le bateau tombe à 0%, les pêcheurs sur le bateau ne seront pas capables de la voir. Vous devrez donc déterminer si les pêcheurs seront guidés à la côte ou non par le Phare. La lumière entre exactement une fois dans le nuage et en sort exactement une fois.

Spécifications d'entrée:

En entrée vous recevrez:

- ***lighthouse***: une liste contenant deux floats, soit les coordonnées du phare
- ***boat***: une liste contenant deux floats, soit les coordonnées du bateau
- ***cloud***: une liste de coordonnées décrivant la forme du nuage (donc un tableau 2D de floats)

Spécifications de sortie:

En sortie vous devrez fournir:

- une string affichant "OUI" si les pêcheurs sont saufs ou "NON" s'ils devront se débrouiller sans le phare

Exemple d'entrée:

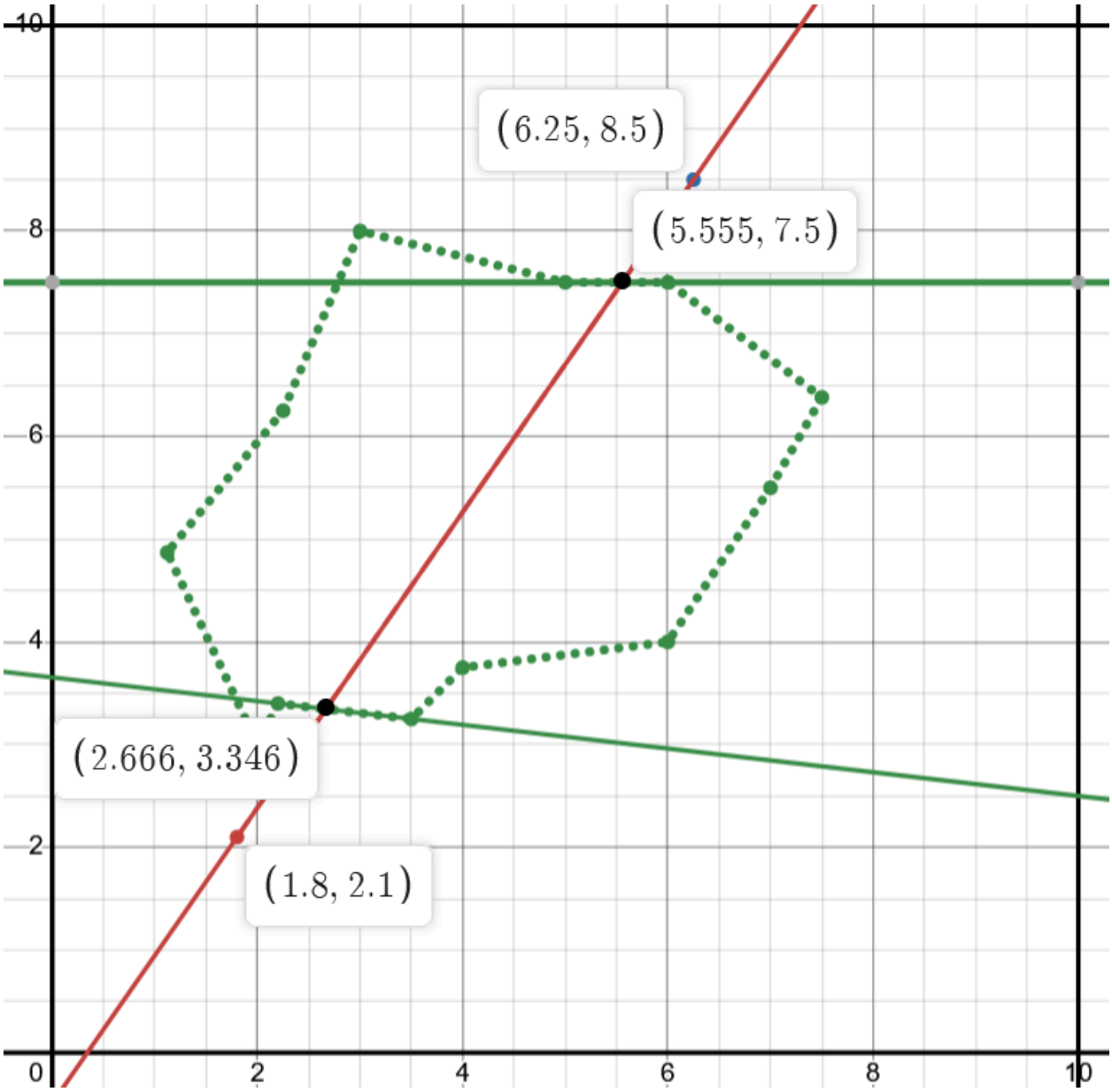
```
lighthouse:[1.8, 2.1]    boat:[6.25, 8.5]
cloud:[[2,3],[2.2,3.4],[3.5,3.25],[4,3.75],[6,4],[7,5.5],[7.5,6.38],[6,7.5],[5,7.5],[3,8],[2.25,6.25],[1.12,4.87]]
```

Exemple de sortie:

```
"NON"
```

Explication de la première sortie:

En calculant les intersections de droite aux endroits où la lumière rentre dans le nuage, on obtient les points (5.555, 7.5) et (2.666, 3.346). La distance entre les deux points est d'environ 5.06 km, ce qui fait tomber l'intensité de lumière à 0% avant qu'elle n'atteigne le bateau. Ainsi, les pêcheurs ne perçoivent pas la lumière et la réponse est "NON". (voir image ci-dessous)



Note en bas de page:

L'absorption de la lumière est en réalité exponentielle, mais c'était plus simple ici de la traiter linéairement.

Mastermind (MM)

MM: Master of the mind (100 points)

Le jeu de mastermind est un jeu de stratégie dans lequel vous souhaitez découvrir la combinaison établie par le mastermind. La combinaison à deviner est une combinaison des 4 de long choisie parmi 6 symboles. Dans la combinaison, le même symbole peut se retrouver plus d'une fois.

Pour trouver la bonne combinaison, vous pourrez faire quelques essais et pour chacun d'entre eux vous recevrez des informations sur la validité de votre essai. Vous aurez en retour deux types d'informations. Un premier chiffre vous dira combien de symboles sont bien placés et un autre chiffre vous indiquera combien de symboles de votre essai sont valides mais au mauvais endroit.

Si vous faites un essai ayant 1 symbole parfaitement placé et tous les autres symboles mauvais vous recevrez en retour (1, 0). Si vous essayez une combinaison avec 3 symboles valides, mais mal placés, vous recevrez le retour (0, 3). Avec 1 parfait et deux mal positionnés vous recevrez (1, 2). Si vous avez gagné, vous recevrez le retour (4, 0). Les symboles utilisés pour notre version du jeu sont: / # = + - *

Vous aurez un total de 20 essais pour trouver la bonne combinaison!

Spécifications d'entrée:

En entrée, vous recevrez:

- la fonction de vérification à appeler pour recevoir les indices de validation. Vous devrez envoyer vos essais à cette fonction tout en vous servant du retour pour améliorer vos prochains essais. Cette fonction prend en entrée un array de string (les 4 symboles de votre essai)

Spécifications de sortie:

En sortie, vous devez retourner la combinaison gagnante dans un array avec les quatre symboles de la combinaison dans le bon ordre.

L'entrée:

```
verif_function
```

La sortie:

Allez voir l'explication…

Explication de la sortie:

On commence notre premier essai avec la combinaison: - - # # et on reçoit (0, 1) On sait alors qu'un seul des deux symboles est valide et qu'il apparait une fois à une endroit ou il n'est pas actuellement. On fait comme 2e essai: + = = - qui nous donne le retour (1,1) On sait

maintenant qu'un des symboles essayé est le bon au bon endroit et qu'un autre symbole est valide, mais mal placé. On continue le prochain essai avec: / + / - qui nous donne le retour: (1, 1) Exactement comme le retour de la combinaison précédente. Notre 4e essai est = = + - qui nous donne le retour (0, 2) Nous avons maintenant 2 jetons valides mais mal placés. On continue avec l'essai: # = / + qui nous donne (1, 3) On sait qu'on est presque arrivé à la solution parce que tous les jetons sont valides et un d'entre eux est au bon endroit. On essai finalement la combinaison / # = + qui retourne (4, 0). Nous avons donc trouvé la bonne solution!! Voici un tableau résumé des essais et des résultats obtenus.

La bonne combinaison dans ce cas était / # = +

s0	s1	s2	s3	résultat
-	-	#	#	(0, 1)
+	=	=	-	(1, 1)
/	+	/	-	(1, 1)
=	=	+	-	(0, 2)
#	=	/	+	(1, 3)
/	#	=	+	(4, 0)